

Curso Inicial Front-End

Clase 14: Funciones y Métodos de Datos

Módulo 2: Dominio de JavaScript y Lógica de Programación



Presentación

No escribas el mismo código dos veces. En esta clase aprenderemos a usar las **Funciones**, el pilar de la reutilización en programación. Entenderemos cómo encapsular lógica para usarla cuando la necesitemos y dominaremos la diferencia técnica entre **definir** una función (crear la receta) y **ejecutarla** (cocinar el plato).



Objetivos

- Dominar la sintaxis de declaración y expresión de funciones en **JavaScript**.
- Comprender el flujo de parámetros y el retorno de valores (**return**).
- Diferenciar claramente entre la implementación de una función y su llamada o ejecución.



Bloques Temáticos

- Tema 1: Declaración de funciones y expresiones (Arrow functions).
 - ¿Qué es una Función? La Máquina de Café
 - Parámetros vs Argumentos: La Trampa de los Novatos
 - Tipos de Funciones (La Evolución)
 - El Valor de Retorno (`return`): El Paquete de Regalo
 - Scope (Alcance): ¿Dónde vive mi variable?
 - Parámetros Avanzados (No rompas el código)
 - Funciones de Orden Superior (HOF) y Callbacks
- Tema 2: Métodos esenciales de manipulación de Strings y Numbers.

Hasta ahora, nuestro código se ejecuta una sola vez de arriba hacia abajo. Pero, ¿qué pasa si necesitas validar la edad de 5 usuarios diferentes a lo largo de tu programa? Escribir el mismo bloque `if/else` 5 veces sería un desastre de mantenimiento.

Aquí entra en juego el concepto de **Función**. Es el bloque de construcción fundamental en **JavaScript** y la base del principio **DRY** (*Don't Repeat Yourself* - No te repitas).

 **Recurso Oficial:** [MDN Web Docs - Funciones](#)

- Tema 1: Declaración de funciones y expresiones (Arrow functions).

Tema 1: Declaración de funciones y expresiones (Arrow functions)

1. ¿Qué es una función? La Máquina de Café

Imaginemos una **Máquina de Café**. Tú no necesitas saber los cálculos eléctricos ni la termodinámica para usarla. Solo necesitas saber tres cosas: qué ingredientes meter, presionar un botón, y qué vas a obtener al final.

Técnicamente, una Función es un **bloque de código empaquetado en memoria** al que le ponemos un nombre. Solo se ejecutará cuando nosotros decidamos "llamarlo" o "invocarlo".

- **Entrada (Parámetros):** Le damos agua y café molido.
- **Proceso (Cuerpo de la Función):** La máquina calienta y filtra. Este es el código lógico oculto entre las llaves `{ }`.
- **Salida (El Retorno):** La máquina nos entrega una taza de café lista para tomar.

En lenguaje simple: Una función es un bloque de código que puedes reusar muchas veces. Cuando la llamas, la computadora la pone al principio de su lista de tareas (Call Stack) hasta que termina su trabajo.

Traducción Técnica: Una función es un objeto de primera clase que encapsula una serie de instrucciones. Al invocarse, se genera un **Execution Context** (Contexto de Ejecución) en el **Call Stack**. Los parámetros funcionan como

variables locales dentro de ese ámbito. La sentencia `return` especifica el valor que se devuelve al contexto de llamada, destruyendo el contexto local y liberando sus variables de la memoria (salvo en casos de *closures*).

2. Parámetros vs Argumentos: La Trampa de los Novatos

En las entrevistas técnicas, confundir estos dos términos es un error grave. Son cosas distintas según el momento en el que estés:

1. **Parámetro (El "Hueco" vacío):** Es la etiqueta temporal que creas en la "puerta" de tu función cuando la estás inventando. Es como un molde vacío esperando recibir datos. *Ejemplo visual:* En `function sumar(numeroA, numeroB)`, `numeroA` es el parámetro.
2. **Argumento (El Dato Real):** Es el valor matemático o texto verdadero que inyectas en ese hueco al momento exacto de ejecutar o llamar a la función en la vida real. *Ejemplo visual:* En `sumar(10, 5)`, el `10` y el `5` son los argumentos.

3. Tipos de Funciones (La Evolución)

Implementar vs. Ejecutar: El Plano y la Casa

Uno de los errores más comunes es creer que con solo escribir la función, esta se activará. 1. **Implementar (Declarar):** Es como dibujar el **plano de una casa**. No puedes vivir en el plano; es solo la instrucción de cómo se debe construir. 2. **Llamar (Ejecutar/Invocar):** Es el acto de **construir y habitar la casa**. Hasta que no escribes `nombreDeLaFuncion()`, el código dentro del plano es invisible para el programa.

```
// 1. IMPLEMENTACIÓN (El Plano)
function cocinarPizza() {
```

```
    console.log("Cocinando...");  
}  
  
// El programa llega hasta aquí y no pasa NADA.  
  
// 2. EJECUCIÓN (Habitar la casa)  
cocinarPizza(); // ¡Aquí es donde la pizza se cocina realmente!
```

Función Declarada (El formato clásico)

Se crea usando la palabra reservada `function`. Tiene una "magia" especial llamada **Hoisting** (Elevación invisible). **JavaScript** lee todo tu archivo antes de ejecutarlo y mueve todas estas funciones al techo del archivo. ¡Por eso puedes llamarlas antes de siquiera haberlas escrito!

```
/* Paso 1: JS la movió al techo, así que podemos ejecutarla (invocarla) antes. */  
/* Observa el uso de los paréntesis () para ejecutarla, inyectando el Argumento "Ana". */  
saludarUsuario("Ana");  
  
/* Paso 2: La declaración formal de la función, con el Parámetro 'nombre'. */  
function saludarUsuario(nombre) {  
    console.log(`Hola, ${nombre}!`);  
}
```

Funciones de Flecha (Arrow Functions)

Llegamos al estándar moderno (desde el año 2015). En **React.js**, verás estas funciones el 99% del tiempo. Eliminamos la palabra `function` y usamos una "flecha gorda" `=>` para apuntar hacia la lógica.

Mapeo de Sintaxis (Cómo leerla): `const nombre = (parametros) => { código a ejecutar }`

A diferencia de las funciones clásicas, las Arrow Functions no tienen Hoisting. Si intentas llamarlas antes de crearlas, la pantalla arrojará un error mortal.

```
/* Modo de varias líneas (Necesita abrir llaves {} y usar la palabra  
return explícita) */  
const validarEdad = (edad) => {  
  if (edad >= 18) {  
    return "Acceso Concedido";  
  } else {  
    return "Acceso Denegado";  
  }  
};  
  
/* Modo Minimalista de 1 sola línea (El "Return" es automático, no se usan  
llaves {}) */  
/* Es ideal para cálculos matemáticos rápidos */  
const calcularDoble = numero => numero * 2;
```

4. El Valor de Retorno (`return`): El Paquete de Regalo

El `return` es la palabra más importante de toda función matemática o lógica. Si tu función calcula algo (ej: sumar $5 + 5$), pero olvidas poner la palabra `return 10`, es como si la máquina de café procesara el agua pero se guardara la taza adentro. ¡Tú necesitas que el dato salga al exterior para guardarlo en una variable, leerlo u operarlo!

⚠ IMPORTANTE

El Freno de Mano Mortal: Cuando el motor de **JavaScript** choca contra la palabra `return`, la función muere y se expulsa instantáneamente. Cualquier línea de código que escribas **debajo** de un `return` jamás será leída ni ejecutada por el sistema.

5. Scope (Alcance): ¿Dónde vive mi variable?

Las variables que creas (`let` o `const`) tienen fronteras de vida útil llamadas **Scope**.

- **Ámbito Global (La Calle Pública):** Si declaras `let iva = 0.21;` suelta al inicio de tu archivo, todo el mundo (cualquier función) puede leerla y modificarla.
- **Ámbito Local (El Cuarto Privado):** Si declaras una variable *dentro* de las llaves `{ }` de una función, esa variable nace y **muere** allí mismo. Ninguna otra función puede verla. Esto es vital por seguridad, para no "contaminar" el archivo general con variables temporales.

Ejemplo visual de Scope en Código:

```
// Variable Global: Vive afuera, todos pueden usarla
let nombreTienda = "SuperMercado JS";

function calcularDescuento() {
  // Variable Local: Nace aquí adentro, y morirá al llegar a la llave de
  // cierre.
  let descuentoSecreto = 50;

  // La función SÍ puede leer lo que está afuera (Global)
  console.log(`Bienvenidos a ${nombreTienda}. Tienes un descuento de
  ${descuentoSecreto}`);
}

calcularDescuento();

// ✗ ERROR CRÍTICO: Intentamos leer la variable local desde afuera.
// JS nos dirá "descuentoSecreto is not defined", porque esa variable ya
// murió.
console.log(descuentoSecreto);
```


6. Parámetros Avanzados (No rompas el código)

Parámetros por Defecto (Plan B)

¿Qué pasa si el usuario olvida enviarte un argumento? En lugar de que tu matemática explote porque multiplicaste por `undefined`, puedes preparar tu función con un plan B ("Por Defecto"):

```
// Si no me envían la 'altura', asumiré por mi cuenta que vale 10.
function area(base, altura = 10) {
    return base * altura;
}

console.log(area(5, 2)); // Resultado: 10 (El usuario envió el 2, así que ignora el 10).
console.log(area(5));    // Resultado: 50 (El usuario olvidó el segundo dato, así que usa el 10 por defecto).
```

El Operador Rest (...): La bolsa de las sobras

A veces no sabes cuántos argumentos te va a pasar el usuario (ej: un carrito de compras que puede tener 1 o 100 productos). El operador `...` junta todos los argumentos infinitos y los mete en un Array automático.

```
function sumarTicket(...precios) {
    // 'precios' ya no es un número suelto, ¡ahora es una Lista/Array matemático!
    console.log(precios);
}

sumarTicket(10, 50, 99, 120); // Consola imprimirá: [10, 50, 99, 120]
```

7. Funciones de Orden Superior (HOF) y Callbacks

En **JS**, las funciones son "Ciudadanos de Primera Clase". Esto significa que no son especiales; una función puede ser tratada como un dato cualquiera, guardada en una variable, o incluso *enviada como parámetro adentro de otra función*.

- **Callback:** Es una función que tú le pasas a otra función como si fuera un ingrediente, con la promesa de que la "llame después".
- **Higher-Order Function (HOF):** Es simplemente la función grande que recibe a ese callback. Las usarás todos los días en **React** (ej: los eventos de clic o los bucles modernos).

Ejemplo visual de un Callback:

```
/* Esta función (Callback) solo sabe saludar */
const saludarAmigo = (nombre) => {
  console.log(`¡Qué tal, ${nombre}!`);
};

/* Esta es la función principal (HOF) que acepta una función intrusa como
parámetro */
function procesarUsuario(nombreUsuario, funcionIntrusa) {
  // Hace cosas...
  console.log("Procesando en base de datos...");

  // Al terminar, ejecuta la función que le pasaron por parámetro
  funcionIntrusa(nombreUsuario);
}

/* Le inyectamos la función 'saludarAmigo' (Sin paréntesis, porque no
queremos ejecutarla todavía, solo se la entregamos) */
procesarUsuario("Lucas", saludarAmigo);
```

Tema 2: Métodos esenciales de manipulación de Strings y Numbers

Imagina que estás organizando los papeles y formularios de una oficina:

- **Limpiar los bordes:** Si un papel viene con espacios vacíos o arrugas al inicio, los recortas.
- **Estandarizar copias:** Si alguien escribió su nombre en mayúsculas y otro en minúsculas, pasas todo a un mismo formato para poder archivarlo sin errores.
- **Contar elementos:** Revisas cuántas letras tiene un código de seguridad.

Eso hacen los métodos de manipulación: transforman, limpian o analizan datos crudos (textos y números) para que la computadora pueda trabajar con ellos de forma prolija.



Analogía: Strings (Cadenas de Texto) como una Cinta de Texto

Piensa en una tira de papel con letras impresas:

- Acción: Cortar un pedazo de la cinta con una tijera (**.slice**).
- Acción: Buscar si una palabra específica aparece en la cinta (**.includes**).
- Acción: Dividir la tira en varios pedazos cada vez que encuentres una coma (**.split**).



Analogía: Numbers (Números) como una Calculadora de Oficina

Cuando trabajas con presupuestos o tickets:

- Acción: Recortar los decimales de un precio largo para que quede presentable en el ticket (**.toFixed**).
- Acción: Leer el texto de una factura que dice "42px" o "12.50\$" y extraer mágicamente solo el número limpio (**parseInt / parseFloat**).

¿Qué son los Métodos de Strings y Numbers en Programación?

Los métodos son funciones integradas en JavaScript que nos permiten manipular cadenas de texto y valores numéricos.

Detrás de escena (Autoboxing): Aunque los strings y numbers son datos simples (primitivos), JavaScript los envuelve temporalmente en un "objeto" automático para permitirte usar estos métodos y, apenas termina, destruye esa envoltura para no gastar memoria. Los strings son además **inmutables**: ningún método cambia el texto original, siempre devuelven uno nuevo.

Métodos de Strings (Cadenas de Texto)

***¿Cuándo usarlos?** Siempre que necesites validar correos, limpiar datos introducidos por usuarios en formularios, buscar palabras clave o cortar fragmentos de texto.*

```
// 1. Búsqueda: .includes() -> Devuelve true o false si el string contiene el valor pasado por parámetro.  
const frase = "Aprender JavaScript es genial";  
console.log(frase.includes("JavaScript")); // true
```

```
// 2. Limpieza: .trim() -> Elimina espacios y saltos de línea al inicio y final, además considera los espacios vacíos " " como strings inválidos "".
const emailSucio = "  usuario@correo.com \n";
console.log(emailSucio.trim()); // "usuario@correo.com"

// 3. Modificación: .toUpperCase() convierte en mayúsculas un string. /
.toLowerCase() convierte en minúsculas un string.
console.log("hola".toUpperCase()); // "HOLA"

// 4. Extracción: .slice(inicio, fin) -> Corta un pedazo
console.log(frase.slice(9, 19)); // "JavaScript"

// 5. División: .split(separador) -> Rompe el texto y lo vuelve un Array
const frutasTexto = "uva,manzana,pera";
let arrayFrutas = frutasTexto.split(",");
console.log(arrayFrutas); // ["uva", "manzana", "pera"]
```

| Métodos de Numbers (Números)

¿Cuándo usarlos? Al calcular precios con decimales, convertir datos que el usuario escribió en la pantalla (que siempre entran como texto) a números reales, o validar si una operación matemática dio un error.

```
// 1. Extracción de enteros: parseInt() -> Lee de izq. a der. e ignora letras del final
console.log(parseInt("42px")); // 42
console.log(parseInt("3.14")); // 3

// 2. Extracción de decimales: parseFloat() -> Mantiene el punto decimal
console.log(parseFloat("12.50€")); // 12.5

// 3. Conversión pura: Number() -> Da NaN si el texto tiene letras
console.log(Number("100")); // 100
console.log(Number("100px")); // NaN

// 4. Formateo: .toFixed(decimales) -> Redondea y limita decimales. ⚠️
¡RETORNA UN STRING!
const precioOriginal = 19.9876;
console.log(precioOriginal.toFixed(2)); // "19.99"
```

```
// 5. Validación estricta: Number.isNaN() -> ¿La operación falló  
catastróficamente?  
console.log(Number.isNaN(0 / 0)); // true (0 dividido 0 no existe)  
console.log(Number.isNaN("hola")); // false (es un texto común, no un  
error matemático)
```

| El Objeto Math

¿Cuándo usarlo? Cuando necesitas herramientas matemáticas que van más allá de una suma o resta básica, como redondear de diferentes formas o generar números al azar.

```
// Redondeos críticos  
console.log(Math.round(4.5)); // 5 (Al entero más cercano)  
console.log(Math.floor(4.9)); // 4 (Fuerza el redondeo hacia abajo)  
console.log(Math.ceil(4.1)); // 5 (Fuerza el redondeo hacia arriba)  
console.log(Math.trunc(4.9)); // 4 (Simplemente borra/mutila los  
decimales)  
  
// Generar un número entero aleatorio entre 1 y 10  
let min = 1;  
let max = 10;  
let numeroAleatorio = Math.floor(Math.random() * (max - min + 1)) + min;  
  
console.log(`Tu número de la suerte es: ${numeroAleatorio}`);
```



Documentación Oficial (Referencias)

- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **React:** <https://es.react.dev/>